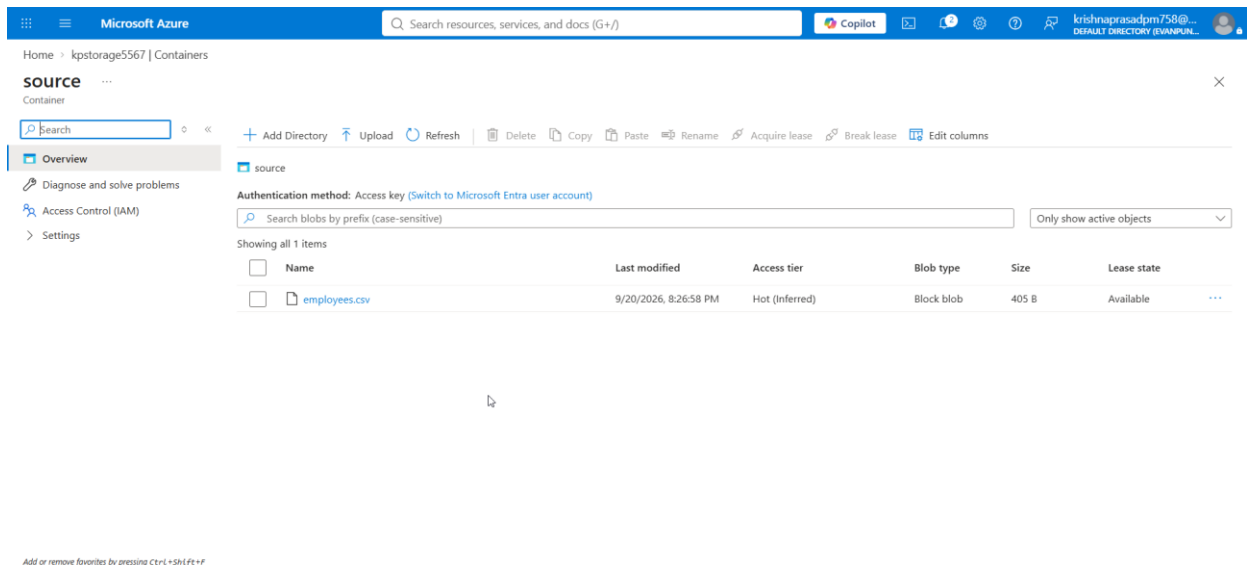


Connection to Databricks from Azure Storage (ADLS Gen2) – Secure Secrets with Azure Key Vault

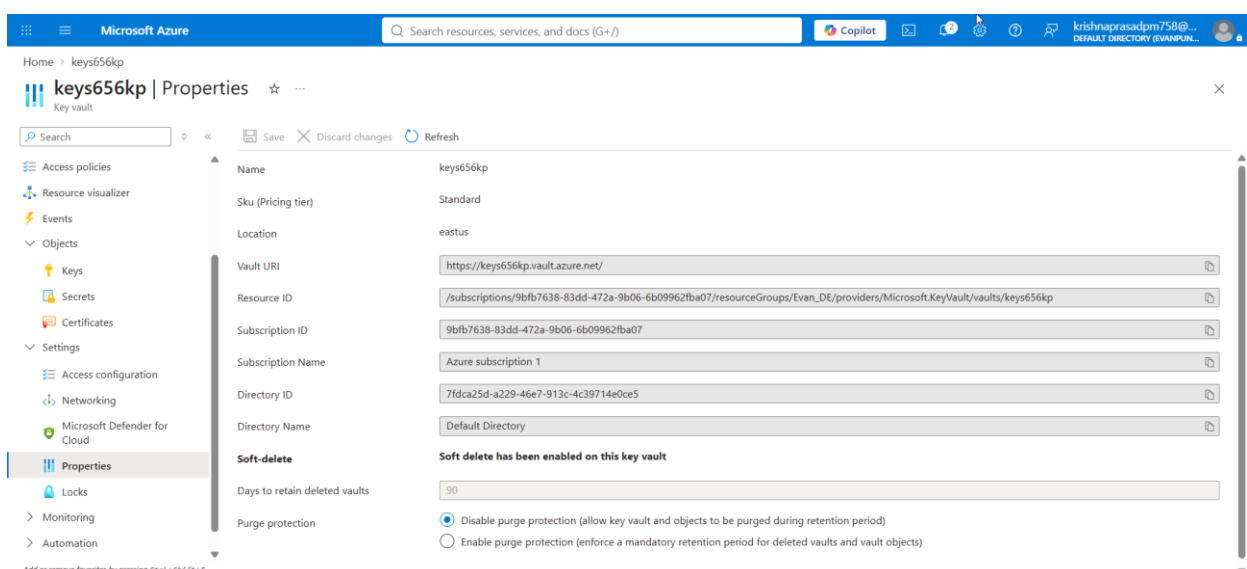
Step 1: Review the Source Data

- Opened storage account `kpstorage5567` and its source container, which holds `employees.csv` — the file Databricks will read.



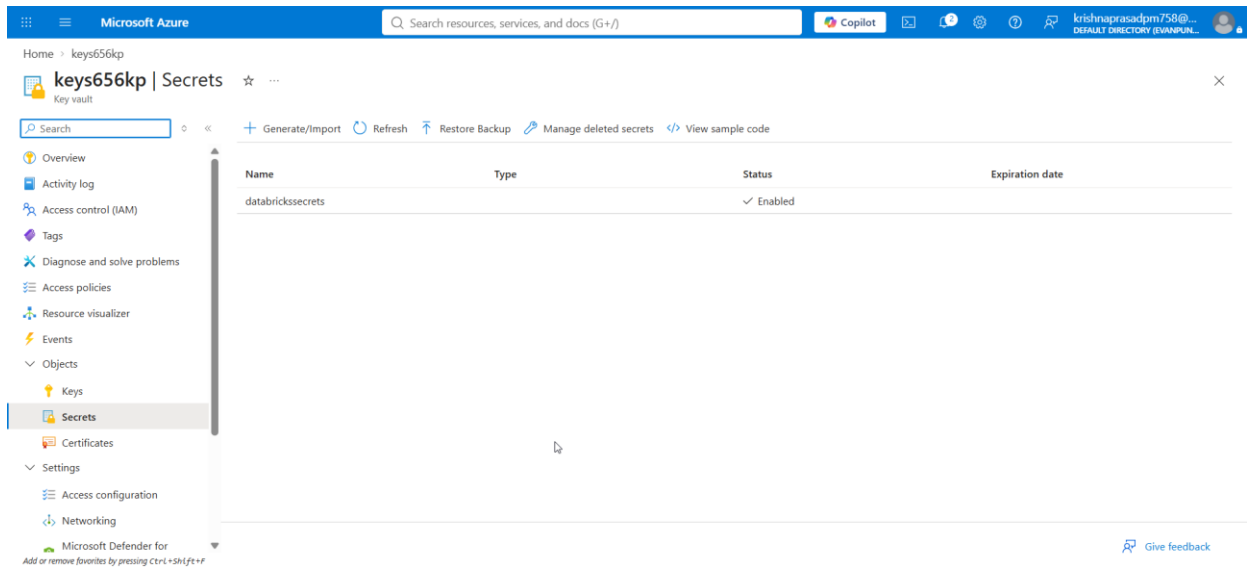
Step 2: Create the Key Vault

- Created a Key Vault named `keys656kp` (Standard tier, East US), with Vault URI `https://keys656kp.vault.azure.net/` and soft-delete enabled — the secure store that will hold the storage account credentials.



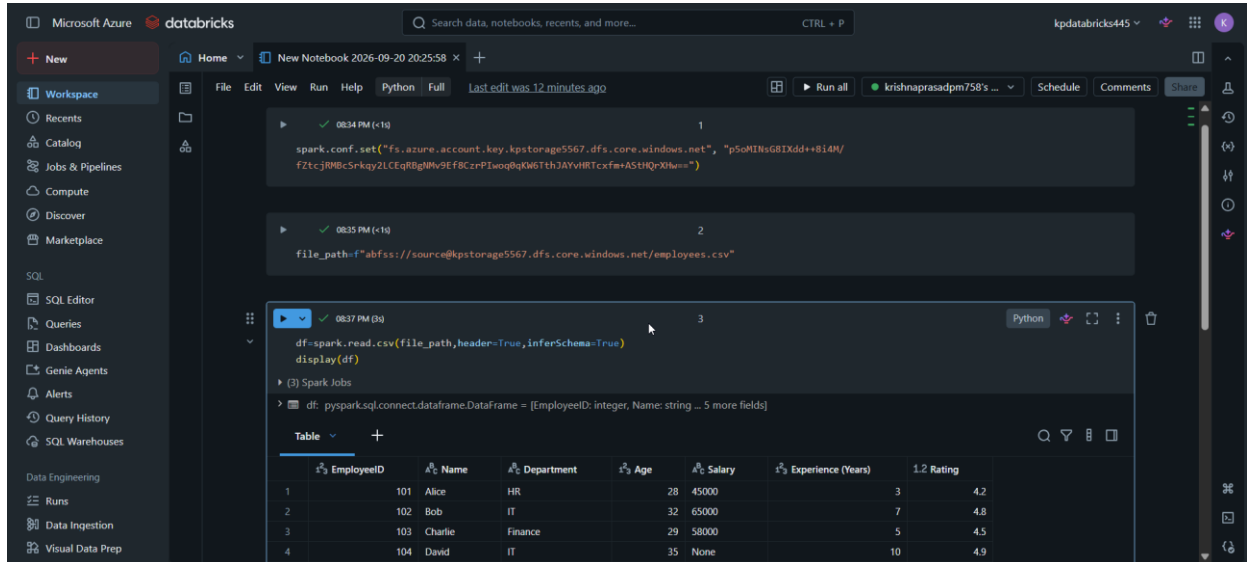
Step 3: Store the Storage Key as a Secret

- In the Key Vault's Secrets page, added a secret named databrickssecrets containing the storage account's access key, with its status showing Enabled.



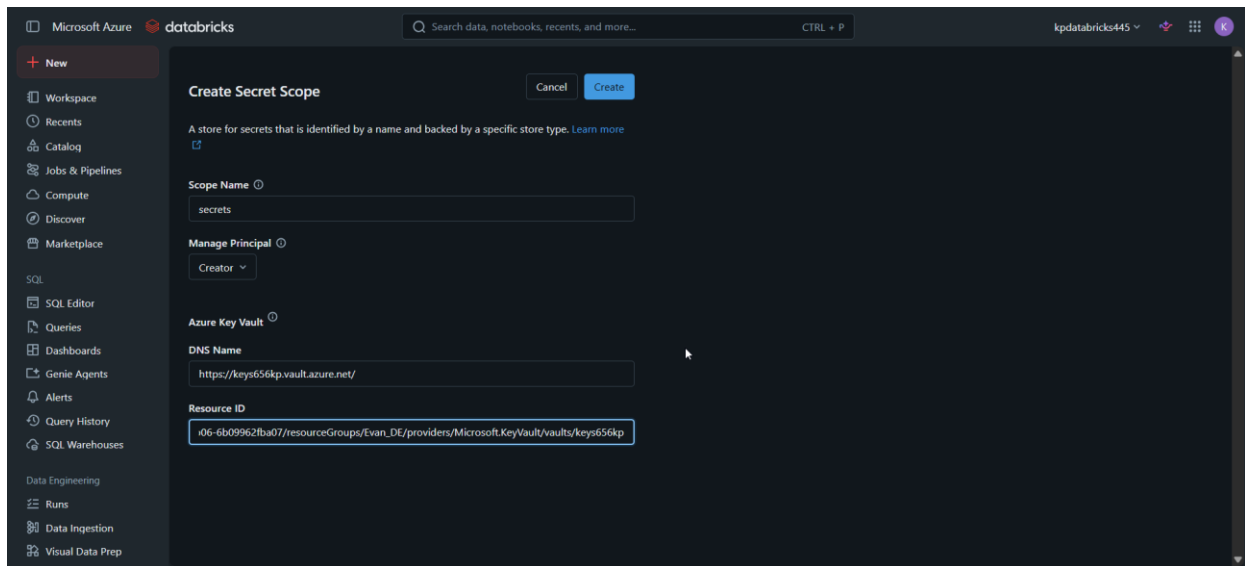
Step 4: Access Storage with a Hardcoded Key (Before)

- In a Databricks notebook, first set `fs.azure.account.key.kpstorage5567.core.windows.net` directly via `spark.conf.set()` with the access key pasted into the code, then read `employees.csv` from `abfss://source@kpstorage5567.dfs.core.windows.net/` into a dataframe and displayed it — exposing the raw key in plain text within the notebook.



Step 5: Create a Databricks Secret Scope Backed by Key Vault

- Created a Databricks secret scope named secrets, linked to the Azure Key Vault via its DNS name (<https://keys656kp.vault.azure.net/>) and Resource ID — so Databricks can pull secrets from the vault instead of storing them in code.



Step 6: Retrieve the Secret Securely

- Replaced the hardcoded key with `secret_name = dbutils.secrets.get(scope="secrets", key="databrickssecrets")`, pulling the value from Key Vault at runtime. Printing `secret_name` returns [REDACTED] — Databricks automatically masks secret values in notebook output, so the key is never exposed.

